

Vortex Gaussians: Real-Time Simulation-Driven Fire and Smoke as Native Gaussian-Splatting Content

ANONYMOUS AUTHOR(S)



Fig. 1. Real-time, simulation-driven fire and smoke rendered as native Gaussian-splatting content. *Left*: a flame driven by nine layered 2D grid-free Gaussian vortex-particle simulations, in which every vortex element is an emissive anisotropic Gaussian (white-hot core $\rightarrow$ orange $\rightarrow$ smoke-darkened tips, with embers). *Middle*: the same solver burning inside a loaded 3DGS scene, from a three-quarter view. The flame and the scene’s splats share one depth order, so the post stands in front of the fire and occludes it per primitive rather than being composited behind it; combustion state lives on the scene’s own splats, so the ring and fuel it has already consumed are charred in place and the plume above is fed by the burning geometry. *Right*: the identical particle set re-shaded as smoke (absorption-dominated compositing with warm underlighting). All results run in real time in a browser.

Fire and smoke remain absent from Gaussian-splatting (GS) scenes: physics-integrated GS handles solids and liquids, GS fire methods only reconstruct observed flames, and the one prior system we are aware of that simulates new fire composites ray-marched volumes over an opaque backdrop at seconds per frame. We present the first system we are aware of that generates fire and smoke by forward physical simulation in real time and renders them as *native* Gaussian-splatting content. Our key insight is that Lagrangian vortex particles and 3D Gaussians are the same class of point primitive: one particle set serves simultaneously as simulation state and render primitive—position to Gaussian mean, temperature to emissive blackbody color, density to opacity, velocity gradient to anisotropic covariance—with no voxelization, meshing, or simulation-to-renderer conversion anywhere, and, unlike vortex-in-cell or hybrid solvers, no background grid on the simulation side either: flame, smoke and embers are drawn by one shader with a single Gaussian footprint, the same one used for the scene’s splats. A grid-free Gaussian vortex-particle solver drives them: each splat carries a Lamb–Oseen vortex core sharing its mean and kernel scale, so velocity, its analytic gradient, and temperature are closed-form functionals of the same Gaussian set the rasterizer draws, accelerated by a Barnes–Hut treecode over moment-matched super-Gaussians whose top levels double as the directable global field. No field state exists anywhere off the particles—no Poisson solve, indeed no linear solve of any kind (the advecting field itself is not pressure-projected; §4.1). One depth-sorted emission–absorption rasterizer composites flame, smoke, embers, and reconstructed 3DGS scenes with per-primitive mutual occlusion, so a foreground object correctly occludes the flame. Combustion state further lives directly on the scene’s splats: the flame ignites the reconstruction, fire spreads and chars in real time, and burning geometry re-seeds the solver—a bidirectional simulation–scene loop. Our single-file WebGL2 prototype runs its shipping configuration at ~ 60 FPS, and a dense 12.4k-Gaussian quality preset at 24 FPS (1280 $\times$ 800, simulation single-threaded in CPU JavaScript), and ships a deterministic-replay harness that makes every reported number bit-reproducible. We measure what every approximation costs: the treecode against the exact Biot–Savart sum it replaces ($\leq 1.5\%$ velocity error at the accuracy setting, faster than the exact sum at every particle count we can

ship), grid freedom against the retained vortex-in-cell baseline ($4.1\times$ its simulation step at matched counts), and the slice stack against a true 3D vortex-particle solver built inside the same prototype and rendered through the same pipeline ($2.8\times$ the stack’s step at matched budget)—so what each approximation gives up is identified rather than asserted.

CCS Concepts: • **Computing methodologies** $\rightarrow$ **Physical simulation**; *Rasterization*; *Image-based rendering*.

Additional Key Words and Phrases: Gaussian splatting, fire, smoke, vortex particle methods, Biot–Savart, Barnes–Hut treecode, real-time simulation, emissive media, volumetric rendering

1 Introduction

Radiance-field capture has made it routine to bring real environments into the computer as 3D Gaussian-splatting (3DGS) scenes that render photorealistically in real time [?]. A growing body of work now asks how such scenes can be brought to *life*: physics-integrated Gaussian methods animate elastic solids and liquids by attaching continuum solvers to the reconstructed primitives [????]. Yet the most visually iconic dynamic phenomena—fire and smoke—remain conspicuously missing from this ecosystem. Existing Gaussian representations of flames are *inverse*: they reconstruct an observed fire from video [??] and can only replay it, as do neural volumetric methods for smoke and fluids [???]. The one system that synthesizes new fire in reconstructed GS scenes, FieryGS [?], runs an offline 256^3 Eulerian solver and ray-marches the resulting volume over the GS scene treated as an opaque backdrop, at seconds per frame. We do not claim a like-for-like speedup over it—its 256^3 grid carries three orders of magnitude more degrees of freedom than our slice stack, and we have not run it—but the gap in *operating regime*, offline versus interactive, is what motivates a differently-shaped design.

This gap is not accidental. Gaseous combustion has neither a surface to mesh nor persistent material points to track, so the standard

physics-to-GS recipes (bind Gaussians to a deformation field; skin them to simulation particles) do not directly apply, and volumetric emission seems to demand ray marching.

Our starting point is an observation about *representation*: a Lagrangian vortex particle and a 3D Gaussian are the same kind of object. Both are unordered point primitives carrying a position and a handful of per-point attributes; both live in flat GPU buffers; and both are consumed by a sort-and-blend operator (Biot–Savart summation over particles; front-to-back alpha compositing over splats). We exploit this isomorphism literally: **one particle set is simultaneously the simulation state and the render primitive**. Each vortex particle carries circulation Γ , temperature T , and density d ; the renderer reads the very same particle as an emissive anisotropic Gaussian whose mean is the particle position, whose color is a blackbody function of T , whose opacity follows d , and whose covariance is evolved by the local velocity gradient. No voxelization, no marching cubes, no simulation-to-renderer resampling of any kind separates physics from pixels. In the spirit of PhysGaussian’s “what you see is what you simulate” [?], our principle is *what you simulate is what you splat*.

For dynamics we revisit a production idea. Industrial Light & Magic’s film fire system [?] showed that a stack of independent, camera-facing 2D slice simulations, driven by a coarse directable 3D pass, is perceptually sufficient for even hero-shot fire—but it ran on a GPU farm at ~ 20 s per slice per frame, at slice resolutions four orders of magnitude above ours. We do not claim to make that system real-time; we make an interactive-scale instance of its architecture fully Lagrangian and grid-free, which changes it structurally: each slice runs a 2D *Gaussian vortex-particle* solver in which buoyancy injects circulation through the baroclinic term $\beta \partial T / \partial x$, evaluated from the particles’ own Gaussian temperature mixture; velocity is recovered by Gaussian-regularized Biot–Savart summation under a Barnes–Hut treecode at $O(N_p \log N_p)$, whose truncated top levels serve simultaneously as the coarse directable field that nudges all slices toward one coherent bulk motion. Because the simulation state lives on particles rather than slice rasters, re-orienting the camera-facing stack changes only the *lift* of in-plane coordinates into world space—the slices re-face the viewer at zero re-simulation cost, where raster-state slices must re-simulate per view. The solver state is exactly the render state; there is no Poisson solve, no pressure, and no grid. Because the slices’ particles are already Gaussians, the entire stack—flame, smoke, embers, and the static background scene—feeds a single depth-sorted emission–absorption rasterizer with HDR blackbody shading, so the fire correctly occludes, is occluded by, and locally illuminates the environment.

We validate the system with an interactive prototype implemented as a single-file WebGL2 application whose simulation runs in JavaScript on the CPU, demonstrating that the method is cheap enough for real time even without GPU compute. We further contribute a decoupled per-layer interface that hot-swaps each slice between the physical solver and a procedural curl-noise source at runtime, which we use both as a development methodology and as an ablation instrument.

In summary, our contributions are:

- **A real-time combustion loop between a forward flame simulation and the splats of a captured scene.** Combustion state—heat, fuel, char, glow—lives *directly on the scene’s own Gaussians*: the simulated flame ignites the reconstruction, fire spreads splat-to-splat, charring is written into the splats’ rendered attributes, and burning geometry re-seeds the solver with new plumes (§5.6). The spread physics itself is conventional [?]; what is new is the substrate and the closed loop—because burning geometry feeds the solver rather than only the renderer, ignition can propagate through the simulated flow and not only along the adjacency graph. The loop is made possible by compositing fire and scene as one depth-sorted Gaussian population (§5.5), where prior art composites fire *over* GS scenes as an opaque backdrop [?].
- **Zero-resampling closed-form coupling of solver and renderer for gaseous media.** Each splat carries a Lamb–Oseen vortex core that shares its mean and kernel scale; every field the solver integrates—velocity by Gaussian-regularized Biot–Savart, its analytic gradient driving both the dynamics and the rendered covariance $\dot{\Sigma} = \mathbf{L}\Sigma + \Sigma\mathbf{L}^\top$, temperature as a normalized Gaussian mixture over the same means—is a closed-form functional of that one Gaussian set, with no field state, grid, or resampling pass anywhere between physics and pixels (§3, §4, §5). Gases have neither a rest configuration nor persistent material points, which is precisely what *vorticity-carrying* particles supply; the covariance clamps act on the rendered quantity—the rest-configuration-free counterpart of covariance-from-deformation [?].
- **A unified Gaussian hierarchy: one tree, three roles.** The Barnes–Hut tree over moment-matched super-Gaussians is simultaneously the $O(N_p \log N_p)$ far-field accelerator, the directable coarse field that couples the slices (its merged top levels, plus ground-mirror images providing the no-penetration boundary), and the low-dimensional handle direction forces attach to—replacing both the per-slice grids and the coarse grid solver of layered production fire with objects of exactly the type the solver and renderer already speak (§4.2). Because the simulation state is particles, the camera-facing slice stack re-orient at zero re-simulation cost (§4.3).
- **A grid-free solver cheap enough to close the loop at interactive rates.** There is no pressure solve and indeed no linear solve of any kind. We claim engineering synthesis, not a fluids advance—the components (Gaussian vortex blobs, treecodes, SPH estimation) are classical—but the composition has properties none of its parts has alone: the solver’s fields are evaluable at any point in closed form, which makes every approximation *auditable* (treecode against the exact sum it replaces, grid freedom against a retained grid baseline, the slice stack against a matched 3D vortex-particle reference, all inside one prototype behind one renderer), and makes runs bit-reproducible from a seed (§7).

We quantify the design space—per-frame cost scaling in slices, particles, and scene splats; treecode vs. exact Biot–Savart summation (accuracy and crossover); a grid-free vs. grid-based A/B of the same

slice dynamics against the vortex-in-cell solver we retain as a baseline; slice orientation; covariance dynamics; and, centrally, the layered approximation itself, measured as energy spectra and cross-plane coherence against a matched true-3D Boussinesq reference—through a deterministic-replay harness that makes every number bit-reproducible from a seed (§7).

2 Related Work

2.1 Fire and Smoke Simulation

Visually plausible gaseous simulation in graphics descends from the unconditionally stable semi-Lagrangian advection of [?], extended with vorticity confinement to fight numerical dissipation [??] and specialized to combustion by [?], whose separate fuel/hot-product phases and blackbody radiance remain the canonical physically based fire model. Heat-threshold fire *spread* over solid substrates—fuel, ignition temperatures, charring—is likewise established, on botanical meshes [?] and at wildfire mesoscale [?]; we adopt this conventional model but carry its state on the splats of a captured scene (§5.6). Real-time grid solvers moved to the GPU early [??].

Vortex methods discretize the vorticity form of the Euler equations on Lagrangian elements, a lineage in graphics that reaches back to hand-tuned vortex fields for film [?] and 2D vortex-particle gases [?—particles [????], filaments [???], or sheets [??—which auto-satisfy incompressibility in reconstruction, need no pressure projection, and preserve turbulent detail with striking economy. Their classical bottleneck is velocity recovery: naive Biot–Savart summation is $O(N_p^2)$, mitigated either by particle–grid hybrids (vortex-in-cell) [?—which reintroduce a background grid—or by treecodes and fast multipole methods [????], which keep every degree of freedom on the particles. [?] brought fast summation to graphics vortex flows with a particle–particle particle-mesh split whose far field lives on a mesh; our far field is instead itself a set of Gaussians (§4.2). Vorticity-based solvers continue to advance, most recently through flow-map formulations—Eulerian [?] and vortex-particle flow maps [?], the latter high-accuracy and offline where we trade accuracy for a real-time rendering-closed loop. We take the treecode branch, precisely because in our system the particles are also the rendered Gaussians: a grid would cap the solver’s effective resolution below what the splats can display, and would reintroduce exactly the field state our representation exists to eliminate. Complementary to resolving vorticity transport, up-res and synthetic-turbulence methods graft procedural detail onto a coarse solve [????]; our curl-noise perturbation (§4.1) is the cheapest member of that family, and everything below the solver’s resolved scales is appearance, not dynamics. Our Gaussian-smoothed elements are classical [????], with the Lamb–Oseen core [??] chosen because its induced velocity and velocity gradient are closed-form—and because the core profile is, literally, the rendered footprint. We thereby inherit the Lagrangian virtue that matters most here: *the solver’s degrees of freedom are already renderable points*. [?] argued that simulation cost should scale with rendered detail and named real-time vortex-based fire an open problem; our system is one answer.

Production layered fire. [?] decompose film fire into a coarse, directable 3D particle pass (a PIC/FLIP variant [?]) and a stack of independent, camera-facing 2D Navier–Stokes slice simulations

(typically $\sim 2048^2$; film examples at 2048×1556 with 32–128 slices), composited near-to-far with Planck blackbody shading. The insight that fine detail perpendicular to the view plane is perceptually invisible justifies 2D refinement. Their slices never exchange simulation state with one another; coherence comes from the shared coarse forcing, splatted with a Gaussian depth kernel spanning ~ 8 slices. The system is offline (~ 20 s per slice per frame on a GPU farm), renders slices as textures composited against holdout mattes, and its stated limitations—fast-moving cameras, flows perpendicular to the view plane, no inter-slice communication—map directly onto the design decisions we revisit. We replace per-slice grid Navier–Stokes with grid-free 2D vortex particles whose Lagrangian state survives slice re-orientation, replace slice textures with Gaussians shared with the environment, and replace the fixed projection kernel with a coarse global *field*—the top levels of the solver’s own treecode—whose influence is a continuously tunable velocity blend. Where their two levels are both grids, ours are both Gaussians.

Layered and billboard volume rendering. Half-angle slicing, volumetric billboards, and spherical billboards render volumes as stacks of blended proxies [??]; games have long shipped 2.5D flipbook and card-based fire. These techniques require the three ingredients our renderer gets for free from splatting: depth-sorted blending, parallax from depth offsets, and camera-facing orientation.

2.2 Gaussian Splatting

3DGS [?] represents radiance fields as anisotropic Gaussians rasterized by tile-based, depth-sorted front-to-back alpha blending, following the EWA volume-splatting lineage [?]; anti-aliased variants [?], exact ellipsoid rendering [?], particle-scene ray tracing [?], and physically based volumetric-primitive transport [?] refine the formulation. [?] in particular show Gaussian mixtures are a sound basis for *emissive* media under ray tracing; we target the rasterized, real-time end of that trade-off. Dynamic-scene extensions obtain motion from data—per-frame optimization [?] or learned deformation fields [?—and thus replay observed dynamics rather than generate new ones.

Physics-integrated Gaussians animate reconstructed scenes forward in time: PhysGaussian evolves Gaussian kernels under MPM continuum mechanics [?], Gaussian Splashing couples position-based fluids with splat rendering [?], Spring-Gaus embeds spring-mass systems [?], and VR-GS makes such coupling interactive [?]. Generative variants infer the material model rather than integrate a chosen one—PhysDreamer distills interaction dynamics from video priors [?], OmniPhysGS learns constitutive models spanning solids and (offline, MPM-simulated) liquids [?]. All target solids or liquids—media with material points or surfaces—and none address optically thin emissive volumes. Closest in spirit, Gaussian Fluids [?] also solves fluids grid-free on Gaussian degrees of freedom—but by the opposite mechanism at the opposite operating point: its Gaussians serve as *basis functions* for a weak-form velocity field, advanced by per-step optimization at 38–228 s per step, with visualization by separate marker and volume passes rather than by the solution Gaussians themselves. We treat Gaussians as explicit strong-form *vortex elements* advanced by classical time integration in real time,

closed into a rendering loop where the same Gaussians are the picture. The two systems agree on the premise—Gaussians are a sound fluid discretization—and disagree on everything downstream of it: weak vs. strong form, optimization vs. explicit integration, three to four orders of magnitude in step time, and whether the solution set itself is what is rendered. We read the agreement as independent evidence for the premise, and our contribution as showing that one Gaussian set can hold *both* ends: solution state and render state at once.

Gaussians for fire. FlameGS [?] and Gaussians on Fire [?] reconstruct emissive flame light fields from images; PINF [?], HyFluid [?], FluidNexus [?], and Vid2Fluid [?] recover smoke or fluid dynamics from video with physics priors, most recently with hybrid physics-optical Gaussian smoke [?]. These are inverse problems. FieryGS [?] is the closest prior system and, to our knowledge, the only prior one that *synthesizes* fire in GS scenes: an MLLM assigns combustion properties to a reconstructed scene, a 256^3 Eulerian solver simulates fire, and a ray-marched volume is composited over the GS scene attenuated as an opaque background, at 2.37 s per frame (0.84 s of which is the fire/smoke ray march). Our system differs on every axis (Table 1): Lagrangian rather than Eulerian simulation, per-primitive depth interleaving rather than backdrop compositing, rasterization rather than ray marching, and real-time rather than offline. FieryGS’s orthogonal capabilities—material inference, charring—are complementary to our pipeline.

3 Overview

Figure 2 summarizes the system. The domain is a stack of N parallel slice layers (default $N=9$) spanning the fire volume. Once per frame we rebuild each layer’s Barnes–Hut tree, merge its top levels into the coarse director field (§4.2), and advance each layer’s 2D grid-free Gaussian vortex-particle simulation (§4.1); every layer exposes its particles through the contract

$$\text{Layer.step}(\Delta t) \rightarrow \{ \mathbf{x}_i \in \mathbb{R}^2, T_i, d_i, \mathbf{u}_i \in \mathbb{R}^2 \},$$

i.e., in-plane position, temperature, density, and velocity. The renderer is the *only* consumer of this contract: it lifts each particle into world space using the layer’s slice frame (§4.3) and instances one emissive Gaussian per particle (§5). Because the contract is source-agnostic, any layer can be hot-swapped at runtime between the physical solver and a cheap procedural curl-noise plume [?—a property we exploit for incremental system verification and for ablations (§7).

Scope of the grid-free claim. The claim has six honest edges, stated once here. (i) The 2.5D slice decomposition remains: grid-free refers to how each slice’s fields are *represented*, not to the dimensional reduction itself, which is a modeling choice whose price we measure in §7. (ii) The direct buoyancy force is still non-solenoidal (§4.1). (iii) The Barnes–Hut tree, the Morton sort behind it, and the fire-spread spatial hash are *index structures* over the Gaussians—rebuilt from particle state each frame, carrying no independently evolved field; nothing is ever sampled onto them and back. They are the analogue of a renderer’s BVH, not of a solver’s MAC grid. (iv) Velocity induction uses each particle’s isotropic core σ_i , for which the Biot–Savart integral has a closed form; the rendered covariance Σ_i is that core’s

flow-sheared envelope (§5.2), sharing mean and scale but not orientation. An anisotropic induction kernel has no elementary closed form and is future work. (v) Particles carry two temperatures—a linearly cooled dynamics temperature driving buoyancy and baroclinic injection, and a nonlinearly cooled color temperature driving shading (§4.1)—so the mixture the solver differentiates and the colors on screen are two scalar fields over the *same* means and kernels, not one field. (vi) Circulation Γ_i is solver state the rasterizer never draws directly; it reaches the screen only through the velocities and covariances it induces. The defensible identity is therefore: one set of means and kernel scales, every solver field a closed-form functional of it, and no intermediate field representation anywhere—not that every attribute is shared.

4 Two-Level Layered Vortex Simulation

4.1 Per-Slice Gaussian Vortex Dynamics

Each slice simulates buoyant flow in the vorticity–streamfunction form of the incompressible Euler equations. In 2D, vorticity is the scalar $\omega = \partial_x u_y - \partial_y u_x$ and the vortex-stretching term vanishes identically, leaving

$$\frac{\partial \omega}{\partial t} + (\mathbf{u} \cdot \nabla) \omega = (\nabla \times \mathbf{f}_b)_z = \beta \frac{\partial T}{\partial x}, \quad (1)$$

where the right-hand side is the *baroclinic* source obtained by taking the curl of Boussinesq buoyancy $\mathbf{f}_b = \beta (T - T_{\text{amb}}) \hat{\mathbf{y}}$: horizontal temperature gradients at the plume edge inject the counter-rotating vortex pairs that roll rising columns into mushroom caps. This single term closes the temperature $\rightarrow$ buoyancy $\rightarrow$ vorticity loop that gives fire its characteristic motion [??].

Vorticity is carried by the splats themselves. Each particle i —the same object the rasterizer draws—is a Gaussian vortex element [??] with circulation Γ_i and isotropic core radius σ_i , so the slice’s vorticity field is the rendered mixture:

$$\omega(\mathbf{x}) = \sum_j \Gamma_j G_{\sigma_j}(\mathbf{x} - \mathbf{x}_j), \quad G_{\sigma}(\mathbf{r}) = \frac{1}{2\pi\sigma^2} e^{-\|\mathbf{r}\|^2/2\sigma^2}. \quad (2)$$

Every field the solver needs is a closed-form functional of this mixture. Each frame the slice performs four grid-free steps over the 2×3 world-unit domain:

- (1) *Temperature.* Each particle carries a *dynamics* temperature T_i , heated in the source band by $T_i \leftarrow T_i + \Delta t s_0 \exp(-\kappa_s \tilde{x}_i^2)$ and cooled linearly by $T_i \leftarrow (1 - \frac{1}{2}\kappa_c \Delta t) T_i$; advection is free, because the carrier advects (the semi-Lagrangian grid step of Eulerian formulations disappears). The temperature *field* is the Shepard-normalized Gaussian mixture

$$\tilde{T}(\mathbf{x}) = \frac{\sum_j T_j w_j(\mathbf{x})}{\sum_j w_j(\mathbf{x})}, \quad w_j(\mathbf{x}) = e^{-\|\mathbf{x} - \mathbf{x}_j\|^2/4\tilde{\sigma}_j^2}, \quad (3)$$

with kernel width $\sqrt{2} \tilde{\sigma}$ (the geometric mean core of the interacting pair), exact for constant fields and degrading gracefully to $\tilde{T} \rightarrow T_i$ for isolated particles.

- (2) *Baroclinic injection.* Each particle accumulates circulation from the local horizontal gradient of the mixture:

$$\Gamma_i \leftarrow (1 - \lambda_r \Delta t) \Gamma_i + \Delta t \beta \left. \frac{\partial \tilde{T}}{\partial x} \right|_{\mathbf{x}_i}, \quad (4)$$

Table 1. Positioning. “GS-native” (render side) means the phenomenon is rendered as Gaussian primitives depth-interleaved with the scene, not composited over it. “Grid-free solver” means no field quantity is ever discretized on a grid: every degree of freedom the solver integrates is a particle—in our case, the very Gaussian the renderer draws.

	gaseous emissive	forward physics	real time	GS- native	grid-free solver	scene composited
?	✓	✓	—	—	—	matte
PhysGaussian [?]	—	✓	—	✓	—	✓
G. Splashing [?]	—	✓	—	✓	✓	✓
G. Fluids [?]	—	✓	—	—	✓	—
Fire recon. [?]	✓	—	(render)	✓	n/a	—
VR-GS [?]	—	✓	✓	✓	—	✓
FieryGS [?]	✓	✓	—	—	—	backdrop
Ours	✓	✓	✓	✓	✓	✓

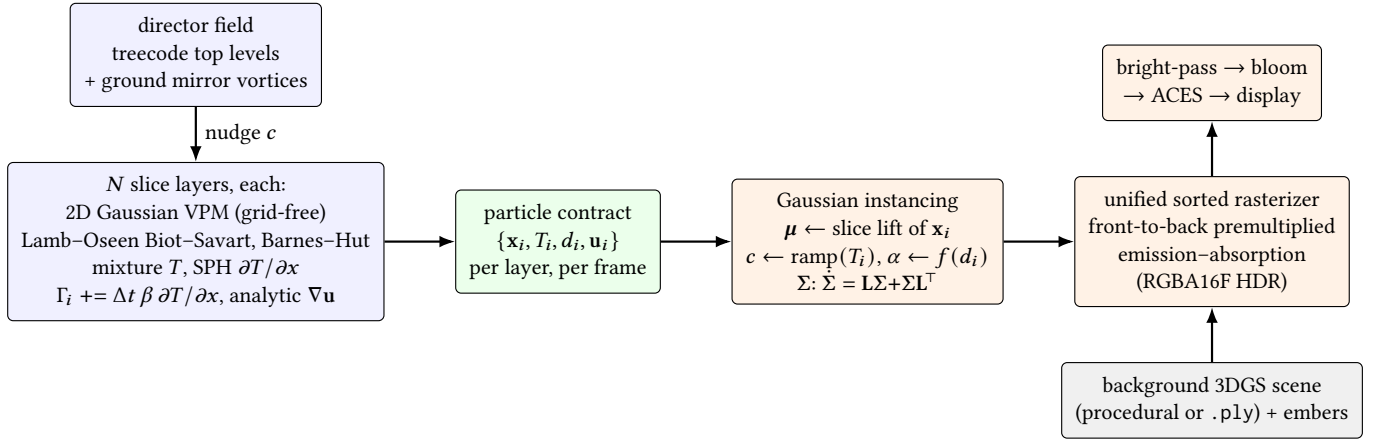


Fig. 2. System overview. A directable coarse field—the top levels of the solver’s own Barnes–Hut tree—drives N per-slice 2D grid-free Gaussian vortex-particle simulations (left, §4). Each slice exposes its particles through a minimal contract (center); the renderer lifts each particle *directly* into an emissive anisotropic Gaussian (§5) and rasterizes all Gaussians—fire, smoke, embers, and the background 3DGS scene—in one depth-sorted emission-absorption pass with HDR post-processing. There is no voxelization or resampling anywhere in the pipeline, nor any solver grid: every field the solver evaluates is a closed-form functional of the rendered Gaussians themselves.

where the gradient is the moment-renormalized SPH difference estimate [?]

$$\left. \frac{\partial \tilde{T}}{\partial \mathbf{x}} \right|_{\mathbf{x}_i} = \frac{\sum_j (T_j - T_i) (\mathbf{x}_j - \mathbf{x}_i) w_{ij}}{\sum_j (\mathbf{x}_j - \mathbf{x}_i)^2 w_{ij}}, \quad (5)$$

which is exact for linear fields under *any* particle distribution—the property that matters at ragged plume edges, where the baroclinic source lives.

- (3) *Velocity and its gradient, in closed form.* The mixture (Eq. 2) induces the Lamb–Oseen velocity [??]

$$\mathbf{u}(\mathbf{x}) = \sum_j \frac{\Gamma_j}{2\pi} \frac{(-r_y, r_x)}{\|\mathbf{r}_j\|^2} \left(1 - e^{-\|\mathbf{r}_j\|^2 / 2\sigma_j^2} \right), \quad \mathbf{r}_j = \mathbf{x} - \mathbf{x}_j, \quad (6)$$

with three properties we use constantly: it is regular at $\mathbf{r}=0$ (self-induction vanishes; particle advection needs no desingularization parameter beyond σ itself); it is solenoidal *analytically, term by term* (each summand is the perpendicular gradient of a streamfunction, so incompressibility of the recovered field holds exactly, with no iterative solve to truncate);

and beyond a few σ it reduces to the point-vortex kernel, so the exponential is only ever evaluated in the near field. Its spatial gradient $\mathbf{L} = \nabla \mathbf{u}$ is likewise closed-form and is accumulated in the same summation at marginal cost; it drives both the rendered covariance (§5.2)—replacing finite differences with the exact derivative of the field the particles themselves induce—and, in the 3D reference solver, vortex stretching (§7). The sum is accelerated by a Barnes–Hut treecode (§4.2); an exact $O(N_p^2)$ evaluation of Eq. 6 is retained as a reference path and used to measure treecode error (§7).

- (4) *Advection.* Each particle adds direct buoyancy $\beta_b \tilde{T}(\mathbf{x}_i) \hat{\mathbf{y}}$, a small divergence-free curl-noise perturbation [?], wind, and the director nudge of §4.2, then advects with explicit Euler.

Note that buoyancy appears twice by design, with independent gains: the baroclinic term (Eq. 4, gain β) generates the *rotational* response—edge rolling and mushroom caps—while the direct term (gain β_b) is an additional control force for bulk rise. Decoupling the two is an artistic choice, not a derivation, and it has a consequence

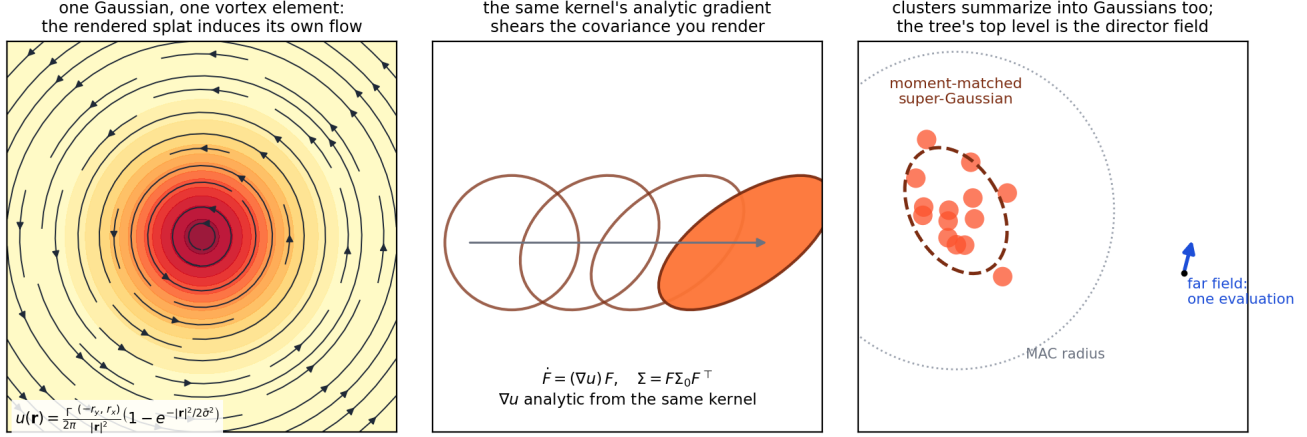


Fig. 3. The identity that organizes the solver: one Gaussian, three closed-form roles. *Left*: the rendered splot is a Lamb–Oseen vortex element—the emissive footprint and the induced velocity field are one object, related in closed form (Eq. 6). *Middle*: the same kernel’s analytic gradient drives the deformation of the covariance that is rendered (§5.2); no finite differencing anywhere. *Right*: treecode clusters are summarized by moment-matched super-Gaussians, so the far field is also a Gaussian evaluation—and the tree’s top level doubles as the director field (§4.2).

we state plainly: $\nabla \cdot (\beta_b T \hat{y}) = \beta_b \partial T / \partial y \neq 0$ in a thermal plume, so while the recovered field of Eq. 6 is solenoidal term by term, the field that actually advects particles is not. With $\beta_b=0.9$ against $\beta=1.4$ this term is the same order as the baroclinic response. We therefore claim that the solver needs no pressure solve—indeed no linear solve of any kind—and not that the advecting velocity is divergence-free.

Viscosity and core evolution (optional). Because each element is a Lamb–Oseen vortex, viscous diffusion has an exact per-particle form: core spreading, $d\sigma_c^2/dt = 2\nu$ [?]. We expose it as an artistic softening control, default off. We use it as a controlled dissipation knob, not as a convergent viscous solver: pure core spreading famously approximates the wrong equation in the large [?], and the splitting schemes that repair it [?] are unnecessary at our 2–4 s particle lifetimes, over which cores grow by at most a factor of two before recycling.

Thermodynamics and lifecycle. Alongside the dynamics temperature, particles carry a rendered *color* temperature, cooled nonlinearly, $dT_i^c/dt = -\kappa (T_i^c)^2$, which concentrates brightness at the hot base and produces the natural fast fade toward the flame tips; density decays exponentially. The two-temperature split mirrors the structure of grid-based formulations (a linearly cooled advected field driving dynamics; a nonlinearly cooled particle temperature driving shading)—here both live on the particles. Particles are recycled when cold, exhausted, or out of bounds, and re-emitted at the source (emission balanced against the cull rate, up to a 1300-particle cap), so the particle count—and hence the Gaussian count—stays bounded.

4.2 Treecode Far Field and the Director Field

Barnes–Hut over super-Gaussians. Evaluating Eq. 6 exactly is $O(N_p^2)$. We accelerate it with a Barnes–Hut quadtree [?], rebuilt from scratch each frame (Morton-sorted, so leaves are contiguous in memory; construction is a fraction of a millisecond and determinism is trivial). Each node aggregates its subtree into a moment-matched

super-Gaussian: signed total circulation, an $|\Gamma|$ -weighted centroid (with a small uniform floor that keeps centroids defined when signed circulations cancel), a signed dipole moment that restores the leading effect of such cancellation—our emission spectrum is near zero-mean in Γ , so monopoles alone systematically underestimate the far field—and a matched core $\sigma_c^2 = \sum_j |\Gamma_j| (\sigma_j^2 + \|\mathbf{x}_j - \mathbf{x}_c\|^2) / \sum_j |\Gamma_j|$ that absorbs the subtree’s spatial dispersion. A far node is thus itself a Gaussian vortex element and is evaluated by the same closed form (Eq. 6); the multipole acceptance criterion $\|\mathbf{x} - \mathbf{x}_c\| > \max(s/\theta, \frac{3}{2}\sigma_c)$ (node size s , opening angle θ) additionally keeps clusters outside their own cores, where the single-Gaussian surrogate is valid. We report measured relative L_2 velocity error against the exact sum across θ rather than an analytic bound (§7)—the empirical practice of vortex treecodes [?]; at the default $\theta=0.7$ the error is $\sim 1.5\%$, below any difference we could discern in informal A/B flips (we have not run a formal perceptual test; §8).

The director field. Independent slices produce N *uncorrelated* fires. Production layered fire solved this with a splot kernel spanning neighboring slices [?]; we instead couple on the simulation side, and the coupler falls out of the data structure we already have. The top levels of the per-layer trees, merged across all slices into ≤ 16 global super-Gaussians—plus their images reflected through the ground plane with negated circulation, which restore the no-penetration floor without any boundary grid—define a global velocity field $\mathbf{U}$ (evaluated monopole-only, averaged over layers, plus a bulk-buoyancy term from the clusters’ mixture temperature). $\mathbf{U}$ represents the directable bulk motion of the whole fire; every particle is nudged toward it:

$$\mathbf{u}_i \leftarrow \mathbf{u}_i + c (\mathbf{U}(\mathbf{x}_i) - \mathbf{u}_i), \quad c \in [0, 1]. \quad (7)$$

$c=0$ leaves slices fully independent (each layer visibly “its own fire”); $c=1$ slaves all slices to the shared field (one coherent but overly

smooth column); the default $c=0.5$ retains per-slice turbulent detail on top of a shared bulk motion (Figure 9). This is the two-level decomposition of ?—coarse directable motion plus per-slice refinement—with two upgrades: the coupling is a first-class, continuously tunable simulation parameter rather than a fixed projection kernel, and the director level is no longer a grid solver but a handful of super-Gaussians, each an editable vortex element (position, circulation, core size) of exactly the same type the solver and renderer already speak. Both levels of the hierarchy are Gaussians; one tree yields the near-field detail, the far-field acceleration, and the global handle. Direction forces *can* be confined to this low-dimensional level; our prototype currently applies wind and source motion per-particle as well.

Inter-slice vorticity coupling. Equation 7 couples every slice to a shared *global* field, but it does not couple slices to their immediate *neighbors*: two adjacent slices can follow the same bulk motion while their turbulent detail stays uncorrelated, and stacked-2D dynamics structurally drop the out-of-plane transport $\partial^2\omega/\partial z^2$ that would tie them together. We reintroduce this dropped coupling directly, particle to particle. Pairs (i, j) of particles in adjacent slices within kernel range exchange circulation through a symmetric Gaussian weight,

$$\Gamma_i += \delta_{ij}, \quad \Gamma_j -= \delta_{ij}, \quad \delta_{ij} = \Delta t \kappa_z \eta_{ij} (\Gamma_j - \Gamma_i), \quad (8)$$

with $\eta_{ij} \propto G_{\sigma_{ij}}(\mathbf{x}_i - \mathbf{x}_j)$ —a particle-strength-exchange discretization [?] of the transport the stacking deletes. Conservation of total circulation is now pairwise and *exact by antisymmetry* (each pair contributes $\pm\delta_{ij}$), and the grid form’s Neumann ends reduce to the mere absence of a neighbor slice. Production layered fire [?] has no slice-to-slice communication at all—its slices are coupled only through the one-way coarse broadcast—so this term fills an obvious hole, but we are explicit about how modest it is: it is the deleted term’s three-point structure applied explicitly against the previous frame’s circulations, and it carries no $1/\Delta z^2$, so κ_z is a per-frame exchange rate rather than a diffusivity and is not comparable across slice counts. It is *off* in the shipping look (Table 3), a visual preference rather than a verdict: under the spectral protocol of §7 the particle form of the exchange raises adjacent-slice velocity correlation from 0.18 to 0.62 (0.76 combined with coarse coupling, against 0.80 for the matched 3D control; four-seed means), at the price of a measurably smoothed in-plane spectrum—it is a diffusion, and it behaves like one. The grid form of the same operator had measured inconclusively on a snapshot probe, a negative result we retain in §7 as a methodological caution. The two mechanisms are complementary by design—Eq. 7 imposes global bulk coherence, Eq. 8 exchanges local neighbor detail—and must be read against both axes of §7. Crucially, Eq. 8 models the *coupling* that stacking drops; it does *not* recover true 3D vortex stretching $(\omega \cdot \nabla)\mathbf{u}$, whose spectral consequences we treat honestly in §8.

4.3 Slice Geometry

Layer $k \in \{0, \dots, N-1\}$ is assigned the depth offset $z_k = (k/(N-1) - \frac{1}{2})D$ along the stack axis, with a mild per-layer opacity fade $1 - 0.3|z_k|$. We support two slice frames:

Table 2. The complete simulation-to-rendering mapping. Every attribute of the rendered Gaussian is read directly off the simulation particle; there is no intermediate representation.

particle attribute	Gaussian attribute	mechanism
position $\mathbf{x}_i$ + layer k	mean $\boldsymbol{\mu}_i$	slice-frame lift (§4.3)
temperature T_i	color c_i	blackbody ramp (§5.4)
density d_i	opacity α_i	§5.3
velocity gradient $\mathbf{L}_i$	covariance Σ_i	Eq. 9
trajectory history	filament axis	Eq. 10

View-aligned (default). The in-plane axes are the camera’s horizontal right vector and the world up vector, and layers are offset along the horizontal camera-forward direction. Slices reorient continuously as the camera orbits, so the stack always presents its face—the choice of ?, without their per-view re-simulation, because our simulation state lives on particles rather than on slice rasters.

World-fixed. Slices are pinned to a world plane and offset along the world normal. Orbiting reveals the stack edge-on and the volume collapses into stripes (Figure 8).

Viewed edge-on, a world-fixed stack degrades into visible stripes while the view-aligned stack continues to read as a volume (§7). Because re-orientation only changes the *lift* of in-plane particle coordinates into world space—not the simulation state—view-aligned slicing costs nothing and introduces no re-simulation popping; the known failure modes of view-locked slices under fast camera motion [?] are correspondingly milder but not eliminated (§8).

5 Emissive Gaussian Rendering

5.1 Particle-to-Gaussian Mapping

Table 2 lists the mapping. Each particle is instanced as a camera-facing quad whose vertex shader expands the unit square along the Gaussian’s major and minor semi-axes and projects with the full perspective camera; the fragment shader evaluates the Gaussian footprint $g = \exp(-3r^2)$ modulated by a four-octave value-noise FBM detail factor that breaks the characteristic “blobby” look of isolated splats. Instance data is a flat 14-float record (position, two axes, fade, T , d , seed, type)—the direct render-side realization of the simulation contract.

5.2 Flow-Driven Covariance

Isotropic splats read as bubbles; real flames are sheared into streaks and wisps by the flow. Analogously to how PhysGaussian transforms covariances by the deformation gradient [?], we evolve each particle’s in-plane 2×2 covariance directly by the local velocity gradient $\mathbf{L} = \nabla \mathbf{u}$ —evaluated analytically from the induction kernel (§4.1), so the shear that streaks a splat is the exact derivative of the field the splats themselves induce, with no finite-difference truncation and no grid Nyquist limit:

$$\dot{\Sigma} = \mathbf{L} \Sigma + \Sigma \mathbf{L}^\top, \quad (9)$$

integrated explicitly with gain k_Σ , relaxed toward identity at rate k_r , and stabilized in closed form: after a 2×2 eigen-decomposition we renormalize $\det \Sigma = 1$ and clamp the leading eigenvalue $\lambda_0 \leq \kappa_{\max}$

(with $\lambda_1 = 1/\lambda_0$ from the determinant constraint, this bounds the splat’s semi-axis ratio a/b at $\kappa_{\max}$). Under $\text{tr } \mathbf{L} = 0$ exact integration would preserve $\det \Sigma$; the renormalization removes explicit-integration drift. Equation 9 is the differential form of PhysGaussian’s $\Sigma = \mathbf{F}\Sigma_0\mathbf{F}^\top$ [?]—but for a gas, $\mathbf{F}$ and Σ_0 are ill-defined absent a rest configuration, which makes the incremental form the natural one; and because the clamps act on the rendered quantity itself, the unbounded-elongation (“sliver”) failure mode reported for Gaussians under sustained shear [??] cannot occur. The splat’s semi-axes are then $a = s\sqrt{\lambda_0}$, $b = s\sqrt{\lambda_1}$ along the eigenvectors, so vortical shear visibly stretches blobs into licks of flame (Figure 10). Attaching per-particle anisotropy to a coarse solve echoes anisotropic turbulence particles [?], with the anisotropy here doubling as the render primitive’s footprint.

As an optional third tier, a trajectory *filament* stretch elongates each Gaussian along its recent path using an exponentially lagged shadow position $\mathbf{p}^{\text{lag}}$:

$$\mathbf{p}^{\text{lag}} \leftarrow \mathbf{p}^{\text{lag}} + \eta (\mathbf{p} - \mathbf{p}^{\text{lag}}), \quad a = s + k_f \|\mathbf{p} - \mathbf{p}^{\text{lag}}\|, \quad (10)$$

turning the particle set into short path-integrated streaks reminiscent of vortex filaments [??]—an instantaneous-shear complement particularly effective for embers and fast plumes.

5.3 Unified Sorted Compositing

All dynamic and procedural-scene Gaussians—flame, smoke, embers, and the 640-splat background environment—are gathered into one buffer, sorted front-to-back by view depth, and rasterized in a single instanced draw into an RGBA16F target cleared to zero coverage. Fragments output premultiplied $(c_i\alpha_i, \alpha_i)$ and are blended with

$$C \leftarrow C + (1 - A) c_i \alpha_i, \quad A \leftarrow A + (1 - A) \alpha_i, \quad (11)$$

(GL blend ONE_MINUS_DST_ALPHA, ONE)—exactly the front-to-back emission–absorption compositing of the 3DGS forward pass [?]. Opacity is the product of a *footprint* and a per-primitive weight,

$$\alpha_i(r) = \hat{g}(r) \text{ clamp}((\alpha_0 + \alpha_1 d_i) T_i \phi_k, 0, \alpha_{\max}), \quad (12)$$

where r is the radial coordinate of the fragment in the primitive’s own frame, ϕ_k is a per-layer depth fade, and $\hat{g}(r) = (g(r) - g(r_{\max})) / (1 - g(r_{\max}))$ with $g(r) = e^{-\kappa r^2}$ is the Gaussian footprint, pedestal-subtracted and renormalized so that it falls *smoothly* to exactly zero on the quad’s inscribed circle ($r_{\max} = 1$), outside of which fragments are discarded. Subtracting the pedestal at the circle rather than at the quad corner matters: a corner-normalized footprint retains a ~5% alpha step along the quad’s straight edges, which reads as a visible square outline on large splats—the eye is acutely sensitive to straight-line discontinuities. The *same* $\hat{g}$ is evaluated by flame, smoke, ember and scene fragments; only the weight differs. Fire is optically thin and emission-dominated, so its Gaussians carry low $\alpha_{\max}$, accumulating radiance while barely occluding; smoke is absorption-dominated with high α and a density-modulated gray albedo warmed by residual temperature. The distinction matters: with a constant per-fragment α and the footprint demoted to a binary mask, the primitive is a disc sprite rather than a Gaussian, and the quad’s corners become visible whenever the sprite is large on screen. Since emission and absorption are independent per-Gaussian channels, the *same* particle state spans

the fire-to-smoke range under two scalar knobs (emission gain, absorption albedo) with no change to geometry, sort, or solver—consistent with Kirchhoff’s law, hot particles emit and cooled ones absorb—so a single plume can transition from emissive flame at its hot base to absorptive smoke at its cooled tips. Because occlusion is resolved per primitive by the shared sort, the fire correctly sits *inside* the scene: environment splats in front of the flame occlude it, and the flame occludes those behind (Figure 7, left)—in contrast to compositing a ray-marched volume over the scene as a whole [?]. Procedural environment splats share the sorted buffer directly; large *loaded* reconstructions are merged into the same global depth order by bucketed interleaving (§5.5, Figure 7).

5.4 Blackbody HDR Shading and Post-Processing

Fire emission follows incandescence: we map particle temperature through a hand-tuned five-stop ramp approximating Planck blackbody colors [??] (dark ember red $\rightarrow$ red $\rightarrow$ orange $\rightarrow$ yellow $\rightarrow$ near-white), scaled by an HDR emissive gain and modulated three ways: a global low-frequency flicker, a per-splat phase jitter ($\pm 8\%$, seed-keyed), and a fine-scale FBM “lick” factor advected upward through the footprint, which breaks the smooth core into tongues. At the cold end the shading crosses over: below $T \approx 0.3$ each splat fades from emission toward an absorbing, ember-tinted soot gray with higher opacity—the crown of a fire is smoke, not dim flame—and its rendered radius grows by up to $1.5\times$ as T falls, the render-side analogue of thermal expansion (both are pure shading terms; the solver state is untouched). Radiance accumulates far beyond display range, so the composited scene passes through a standard HDR chain: bright-pass extraction, five iterations of half-resolution separable Gaussian blur (bloom), then ACES filmic tone mapping [?], gamma, vignette, and a $1/255$ dither. Combined with the nonlinear particle cooling of §4.1, this reproduces the canonical luminance structure of real flames—saturated white-hot core, graded orange body, smoke-darkened tips—without any texture assets (Figure 12).

5.5 Scene Composition and Secondary Effects

The environment is itself Gaussian content: either a procedurally scattered dark scene (640 flattened ground and enclosure splats) or a reconstructed 3DGS capture loaded from a standard Inria .ply (scales exponentiated, opacities sigmoided, DC plus degree-1 spherical-harmonic color; subsampled to a budget of up to 120k splats). Scene splats are rasterized with the standard EWA forward projection [?]: the full 3×3 covariance is carried per instance, projected to a screen-space conic through the view rotation and perspective Jacobian in the vertex shader, and the conic’s eigenvectors span the quad. Procedural environment splats join the sorted buffer of §5.3 directly, giving per-primitive mutual occlusion with the fire. Loaded reconstructions are merged into the same global depth order by *depth bucketing*: each frame, scene splats are counting-sorted into $K=96$ view-depth buckets in $O(n+K)$, and the renderer walks the buckets near-to-far, interleaving ranges of the (exactly sorted) dynamic stream with ranges of the bucketed scene stream under the shared blend state, coalescing adjacent same-stream ranges to minimize state changes. Order is thus exact within the dynamic stream and bucket-granular between and within the streams—strictly finer

than the per-slice compositing granularity of layered production fire [?]. The bucketing’s measured overhead is small: with the interleave on vs. a draw-after pass at the full test scene (45.8k scene splats over the 9×1300 flame) we measure 30.7 vs. 28.3 ms median render time—a 2.4 ms (8%) premium, with per-seed deltas ranging -0.3 to $+5.6$ ms across runs (Figure 7; measurement protocol in §7).

In-scene ignition. Because the fire shares the scene’s depth order rather than sitting in front of a backdrop, a source can be lit *on a reconstructed surface*: the plume then rises inside the captured geometry and is occluded per primitive by the objects around it (Figure 7, top left). We state the compositing claim precisely: a backdrop system *can* recover much of this occlusion by testing the volume against a per-pixel depth buffer of the scene, and we implement that stronger baseline (Figure 7, top right); what per-pixel depth cannot express is mutual occlusion among *semi-transparent, interpenetrating* primitives—fire splats threading between a foreground post’s splats and translucent smoke wrapping partially charred geometry—where no single depth per pixel exists, so the z -test degenerates to a binary, aliased cut at every soft boundary. Per-primitive interleaving resolves exactly those cases, and in-place authoring falls out of it. Ignition composes with the fire-spread loop of §5.6, which turns a single lit point into scene-wide, multi-source fire.

The fire acts as a light source on the scene through an inexpensive screen-space model: environment splats are warmed by a radial falloff around the flame’s screen position with a floor-proximity boost—a deliberately cheap stand-in for volumetric light transport that nevertheless anchors the flame in the scene (§8). Sparse ember particles (velocity-aligned, twinkling Gaussians in the shared sorted pass), plus two screen-space effects—a mirrored planar ground reflection and an FBM heat-haze refraction above the flame—complete the presentation; all are optional.

5.6 Fire Spread on the Scene’s Own Primitives

Because the environment is itself a set of point primitives, combustion state can live *directly on the reconstruction*: each scene splat carries heat h , fuel f , a burning/charred state, and a glow amount. Adjacency is a spatial hash built once in the reconstruction’s source frame (invariant to scene placement), and per-frame work is restricted to the fire front—burning splats and their neighbors—capped at a fixed budget, plus an $O(n)$ heat-decay scan.

The dynamics close a loop between the simulation and the scene. *Scene ignition*: sampled hot flame particles deposit heat into nearby splats (h decays, so ignition requires sustained heating); past a threshold a flammable splat ignites. *Spread*: burning splats heat unburnt neighbors with a buoyant upward bias, following the standard heat-driven spread models of interactive wood combustion and mesoscale wildfire [??]—the spread physics is deliberately conventional; per-splat fuel supports non-flammable regions. *Consumption*: fuel depletes over a burn time τ ; while burning, an HDR ember-glow term is added in the scene shader, and a char fraction drives the splat’s albedo toward black and reduces its opacity—combustion is written directly into the rendered attributes of the captured primitives. *Scene-to-simulation*: burning splats spawn hot seed particles in the nearest slice (round-robin, budgeted), which enter the temperature mixture (Eq. 3) directly, so ignited geometry grows its *own*

plumes through the full baroclinic loop. The per-frame budgets that bound the loop’s cost are fixed: at most 160 flame-particle heat samples, 48 scene-to-simulation seed emissions, and 1400 frontier splats processed per frame—so spread cost is capped by construction, and the timing we report is a function of the frontier cap, not of how much of the scene burns.

Figure 4 shows the loop end to end on the test scene: a single ignition inside the fire ring spreads through the fuel, the burning splats glow in HDR and darken as they char, and—because they inject temperature and particles into the nearest slice—the plume above the fire is fed by the *scene*, not only by the original emitter. With a ~ 190 -splat active fire line the frame stays comfortably real-time (render 4.6 vs. 3.9 ms with the fire line active vs. quiescent, same machine as Table 4). Whether ignition can cross a fuel gap wider than the adjacency radius we now test directly rather than speculate about: in a two-island configuration (fuel zeroed in an x -interval; floor and wall non-flammable), ignition never crossed gaps of 0.12–0.65 world units (1.1 – $6 \times$ the adjacency radius) in 35 simulated seconds, at winds up to $2 \times$ the default—the burning island chars completely (5 465 splats) while the far island stays untouched. The mechanism is buoyancy: hot particles rise at ~ 1.1 units/s against a lateral wind of 0.35–0.6, so by the time the plume has drifted across even the narrowest gap it is above the far fuel’s height and outside the flame-to-scene heating radius. The crossing we observed in an earlier, more compact configuration reached a *tall* object standing inside the slice slab—consistent with the same mechanism, not an exception to it—so gap crossing in this system is a statement about target geometry, and lateral ember-mediated ignition (embers are currently visual-only) is the natural mechanism to add for it.

We scope the claim precisely: heat-threshold spread with fuel and charring is long established on meshes and voxel grids [??], and FieryGS spreads fire between contacting objects by solid heat diffusion on its occupancy grid, offline [?]. What is new here is the *substrate and the loop*: combustion state carried on the splats of a captured scene, updated in real time, where the forward-simulated flame both ignites the reconstruction and is re-seeded by it. One honest caveat: the loop projects between world space and slice coordinates through the current slice frame, so under view-aligned slicing with a moving camera the spread coupling inherits the view-adaptation of §4.3; the world-fixed mode (or freezing the frame during spread) removes this dependence (§8).

6 Implementation

The entire system—simulation, sorting, rendering, UI—is a single dependency-free HTML file: simulation and per-frame depth sort run in JavaScript on the CPU; rendering is WebGL2 with instanced quads, an RGBA16F HDR target (8-bit fallback), and hand-written camera matrices. We emphasize what this implies: the method needs no GPU compute, no CUDA, and no preprocessing, and still reaches real-time rates (§7). Two named configurations are used throughout and every reported number is tagged with one of them. The **Interactive preset**—the shipping default—is $N=9$ layers $\times$ 600 particles at $\theta=0.9$ (~ 5.4 k flame Gaussians, ~ 60 FPS end to end). The **Quality preset**, used for all accuracy-critical measurements in §7 unless stated otherwise, is $N=9 \times 1300$ at $\theta=0.7$ (~ 11.7 k flame Gaussians), 640

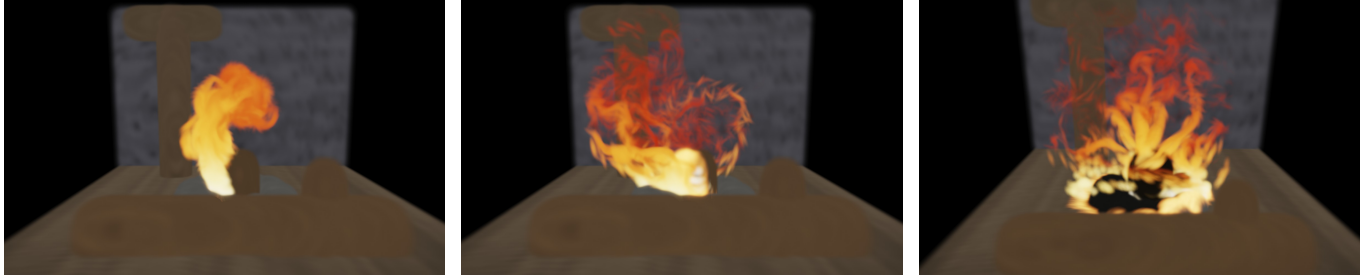


Fig. 4. Fire spread on the scene’s own primitives (§5.6), one continuous simulation on the campfire test scene. A single ignition (4083 splats, radius 0.35) is seeded inside the stone ring at $t=0$; fuel is assigned to every splat below the wall top (41 020 of 45 808), and the stone ring and wall carry none. *Left* ($t\approx 2$ s): the ignition is still local—rear post, fuel logs and ring are intact. *Middle* ($t\approx 16$ s): the front has climbed the post and consumed the nearest log, which is now black; burning splats glow in HDR and inject temperature and particles into the slice solvers, so the plume above is fed by the scene rather than by the emitter. *Right* ($t\approx 56$ s): 12 600 splats charred and 2477 still burning, with char written directly into splat albedo and opacity—the dark remains around the ring, and the charred stub of the post still legible against the wall, are scene geometry rather than smoke. Shown from a raised camera so the char is not hidden behind the flame’s bright core.

Table 3. Key parameters (defaults). World domain 2×3 units.

symbol	meaning	default	section
N	slice layers	9	§4.3
N_p	particles per layer	1300	§4.1
σ_0	vortex core radius	0.05	Eq. 6
θ	treecode opening angle	0.7	§4.2
ν	core-spreading viscosity	0 (opt.)	§4.1
N_{dir}	director super-Gaussians	$\leq 16+16$	§4.2
β	baroclinic gain	1.4	Eq. 4
β_b	direct buoyancy	0.9	§4.1
κ	cooling rate	1.2	§4.1
c	global (coarse) coupling	0.5	Eq. 7
κ_z	inter-slice coupling	0 (opt.)	Eq. 8
D	stack depth	0.7	§4.3
k_Σ	covariance gain	1.3	Eq. 9
κ_{max}	eigenvalue clamp (bounds axis ratio)	3.6	Eq. 9
—	emissive gain / bloom / threshold	1.4 / 1.0 / 0.95	§5.4

procedural environment splats, ≤ 160 embers—about 12.4k sorted Gaussians per frame, plus up to 120k additional splats when a reconstructed scene is loaded (merged into the global depth order by $O(n)$ bucketing; §5.5). Table 3 lists the main parameters. A headless harness drives the demo without a display and reads back probe images for the quantitative measurements in §7. For reproducibility, every stochastic simulation path (emission jitter, ember spawning, initial conditions) draws from a single seeded PRNG stream; combined with a fixed Δt , a run is fully determined by (seed, steps), and an FNV-1a hash over the complete simulation state verifies bit-exact replay (§7).

7 Results and Evaluation

Deterministic replay. All measurements below are taken through a deterministic-replay harness: every stochastic simulation path draws from one seeded PRNG stream, the solver is stepped at a fixed $\Delta t = 1/60$ s, and an FNV-1a hash over the complete particle state (position, circulation, core radius, both temperatures, density,

velocity), embers, and—when the retained grid baseline runs—its coarse field, fingerprints each frame. The treecode is hashed only through its effect on particles: it is derived data, rebuilt from the particles every frame. Two runs from the same seed produce bit-identical state hashes after 120 steps (this holds for both the grid-free solver, whose Morton-ordered build and fixed traversal order are deterministic by construction, and the baseline); a different seed diverges—so every number in this section is exactly reproducible from a seed and a step count.

Performance. Table 4 reports per-frame timings, measured over 90–240 simulated and 60 rendered frames per configuration at 1280×800 on a workstation (AMD Ryzen Threadripper PRO 3945WX; NVIDIA GeForce RTX 3080 via Chrome/ANGLE D3D11; simulation single-threaded in CPU JavaScript), median of five seeds per configuration, all captured in one scripted browser session. Grid freedom is not free, and we price it precisely. Against the retained vortex-in-cell baseline at matched counts ($N=9$, $N_p=1300$, $\theta=0.7$), the grid-free solver’s simulation step costs $2.3\times$ more (29.8 vs. 12.9 ms)—the cost of evaluating $\sim 10^6$ closed-form pair and cluster interactions per frame instead of one 32×48 relaxation per slice—while remaining interactive end to end. At the interactive preset ($N_p=600$, $\theta=0.9$) the premium shrinks to $1.3\times$ (10.4 vs. 7.8 ms) and the full pipeline runs at ~ 60 FPS. What the extra milliseconds buy is measured in the spectral A/B below: velocity detail no longer capped at a grid Nyquist, and a solver whose state is exactly the picture. The same harness times the exact $O(N_p^2)$ Biot–Savart evaluation the treecode approximates (Figure 5). There is no crossover in any regime we can ship: the treecode is already $3.2\times$ faster at 150 particles per slice, and the gap widens to $8.1\times$ at 600 and $17\times$ at 2400 (4.2 vs. 72.2 ms per slice) as the exact sum grows quadratically, while the treecode grows as $N_p \log N_p$ with a measured relative L_2 velocity error of 1.5% at $\theta=0.7$ (0.6% at $\theta=0.5$, 3.3% at $\theta=0.9$, 6.3% at $\theta=1.1$)—the quadratic wall is removed without a grid, and without an unmeasured approximation.

We choose the $N_p=1300$ default deliberately: the criterion is the standard vortex-method overlap condition—mean inter-particle spacing h below the core radius σ_0 , so the blobs tile the plume into a

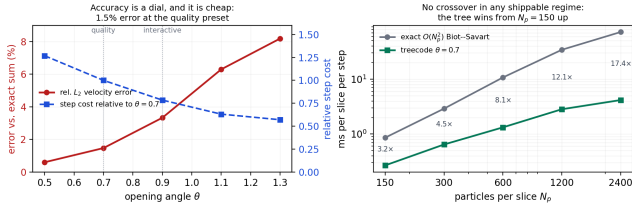


Fig. 5. The treecode, priced against the exact sum it replaces. *Left*: relative L_2 velocity error vs. opening angle θ (error is deterministic and environment-independent), with step cost relative to $\theta=0.7$ measured in one browser session (the absolute anchor is Table 4); the two presets are marked. *Right*: per-slice cost of the treecode vs. the exact $O(N_p^2)$ Biot-Savart sum (3-seed medians, same session as Table 4). The tree is faster at every count we can ship, from $3.2\times$ at $N_p=150$ to $17.4\times$ at 2400—the quadratic wall is removed without a grid.

Table 4. Per-frame cost (ms) vs. slice count N and particles per layer N_p , for both solvers at matched state. sim_{VIC} = retained vortex-in-cell baseline; sim_{VPM} = grid-free Gaussian vortex solver ($\theta=0.7$); sort = gather + exact depth sort; render = GPU-synchronized draw + HDR post (both solver-independent, measured once). Every number is the median of five seeds, all timed in the same Chrome session (V8 JIT, ANGLE D3D11) in one scripted run; simulation is single-threaded CPU JavaScript; every row reproducible from a seed.

N	N_p	Gaussians	sim_{VIC}	sim_{VPM}	sort	render	total $_{\text{VPM}}$ (FPS)
1	1300	1.3k	1.32	3.1	0.40	0.58	4.1 (246)
3	1300	3.9k	4.08	9.6	1.40	4.98	16.0 (62)
5	1300	6.5k	6.91	15.5	2.17	4.95	22.6 (44)
9	1300	11.6k	12.9	29.8	4.24	7.77	41.8 (24)
13	1300	16.7k	18.2	42.9	7.00	10.4	60.3 (17)
17	1300	21.8k	23.6	60.5	10.9	13.7	85.0 (12)
9	300	2.7k	5.38	5.9	0.84	2.91	9.7 (103)
9	600	5.4k	7.85	13.1	2.06	5.55	20.7 (48)
9	2200	15.0k	19.7	42.0	6.54	9.37	57.9 (17)

continuous field—plus image quality (a sparser population no longer reads as a flame). Image quality, not frame time, sets this budget. For calibration of the design point (not of solver efficiency—these systems integrate far larger simulation state), the closest prior systems report 2.37 s per frame (FieryGS [?], RTX 4090D) and 38–228 s per *time step* (Gaussian Fluids [?], RTX 4090); production layered fire required ~ 20 s per slice per frame at $\sim 2048^2$ slice resolution on a GPU farm [?], versus our ~ 1300 particles of core $\sigma_0=0.05$ per slice (the retained grid baseline runs 32×48)—much of the gap is resolution deliberately traded away, the rest is architecture.

Slice orientation. Orbiting the camera toward the slice planes’ edge separates world-fixed slices into the flat sheets they are, and at 90° collapses them into lines, while view-aligned slices continue to read as a volume (Figure 8). We quantify this with an unclipped probe: total HDR energy in the framebuffer *before* tone mapping, swept over 36 azimuths at a fixed simulation state. World-fixed slices lose up to 89% of their radiant energy edge-on (min/max ratio 0.11 across the sweep, coefficient of variation 0.27); view-aligned slices



Fig. 6. Source hot-swap through the layer contract, same renderer and parameters. *Left*: all nine layers driven by the kinematic curl-noise plume—adequate bulk shape, but uniform ascent without vortical rolling. *Right*: all layers driven by the Gaussian vortex solver—baroclinic vorticity rolls the plume edges and produces the characteristic unsteady flame silhouette. Layers can be swapped per-layer at runtime with zero renderer changes.

are azimuth-invariant by construction, and the probe confirms it to numerical precision ($\text{CV} < 10^{-3}$). An earlier peak-brightness version of this probe saturated at the display ceiling and was withdrawn; measuring pre-tonemap energy removes the clipping.

Inter-slice coupling: a cautionary measurement. We attempted to quantify the effect of Eq. 8 with a snapshot probe—the mean Pearson correlation of adjacent slices’ velocity fields. An informal sequential measurement (settings changed on a warm, running simulation) suggested a strong monotonic rise with κ_z ; under the deterministic-replay protocol (per setting: 3 seeds $\times$ 4 snapshots after a 600-frame cold-start warm-up, $c=0$) that trend *does not replicate*: the probe sits at 0.26–0.38 across $\kappa_z \in [0, 1.2]$ with no significant trend, because the shared source geometry already induces strong structural correlation between slices and the probe’s between-setting variance swamps the operator’s marginal effect. We therefore make no coherence claim from *this probe*. The claim that survives it comes from the field-export protocol below, whose ensemble statistics are not dominated by the shared source geometry: there the particle-strength-exchange form of the operator raises adjacent-slice correlation from 0.18 to 0.62 (four-seed means). We report the failed probe deliberately: the replay harness exists so that informal numbers cannot survive into print, and the operator’s grid-era version had passed exactly this kind of informal reading before failing the controlled one.

A true-3D solver through our own pipeline. The comparison we consider most informative is not against another system but against *ourselves without the layering*, because that is the only way to isolate the layered decomposition from every other difference. We therefore implemented a true 3D Gaussian vortex-particle solver inside the shipping prototype and rendered it through the identical path. Everything is held fixed: the world domain (the 3D vortons occupy exactly the slab the slice stack occupies, with the same out-of-slab clamp), the kernel family and core radius σ_0 , the treecode opening angle θ , the particle budget, the baroclinic coefficient, the two-temperature scheme, buoyancy, cooling and Δt , the particle-to-Gaussian mapping, the depth sort, and the rasterizer. Two things differ, and they are what “3D” means: circulation is a three-vector α_i inducing velocity through the 3D Gaussian-regularized Biot-Savart

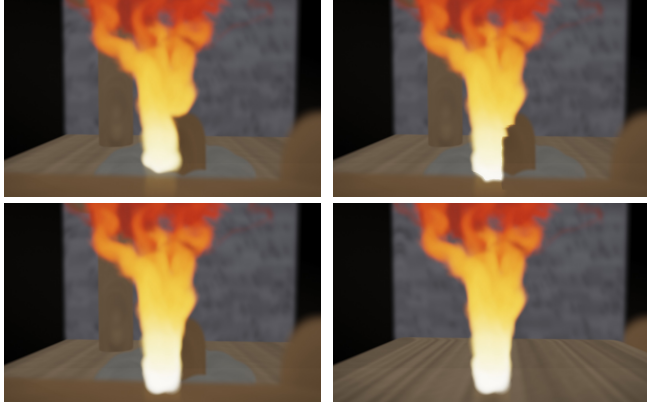


Fig. 7. Per-primitive occlusion with a loaded 3DGS scene (synthetic test reconstruction: plank floor, stone wall, a stone fire ring, fuel logs and a rear post; 45 808 splats), seen from a low, close camera so a fire-ring stone stands directly between the viewer and the white-hot core. Identical simulation state, camera and shading in all four; only the composition differs. *Top left (ours)*: bucketed depth interleaving (§5.5)—the stone’s dark silhouette bites into the core through a soft, correctly blended Gaussian boundary, smoke veils the post at the left, and the flame occludes the wall behind it. *Top right (depth-aware baseline)*: the strongest non-interleaved composition we know how to build: scene splats with footprint opacity ≥ 0.5 write depth, and fire fragments are z-tested against it before compositing over the scene. Opaque occluders now clip the fire, but the decision is binary per pixel—where the flame meets the front stone, the soft Gaussian boundary of the top-left panel collapses into a hard, aliased silhouette, and fire near the interface pops fully in front of or fully behind the surface. No depth threshold fixes this: partial transmittance through a semi-transparent splat is not expressible as a z-test. *Bottom left (backdrop baseline)*: the scene is sorted *correctly* among itself but composites as a whole behind the flame—the analogue of compositing a ray-marched volume onto a reconstructed backdrop [?]. The flame is plausible in isolation yet sits in front of every primitive: the stone’s silhouette vanishes from the core entirely. *Bottom right (unsorted)*: additionally dropping the intra-scene sort—the floor and wall, drawn early in file order, exhaust the alpha of pixels that belong to the post, ring and fuel, which disappear. We show it only to separate the two failures; the depth-aware panel is the fair comparison.

kernel (an erf-form closed expression [?]), and particles carry the vortex-stretching term ($\alpha \cdot \nabla$) $\mathbf{u}$ —evaluated analytically from the same kernel’s gradient—which the stack structurally cannot have. A retained 3D vortex-in-cell solver provides the corresponding grid-based control.

The cost of the layered decomposition is therefore measurable directly. At matched budget ($\sim 5.4\text{k}$ particles), the 3D solver’s simulation step is $2.8\times$ the stack’s (27.5 vs. 9.9 ms); switching stretching off changes its cost by nothing measurable, so the ratio prices dimensionality, not the ablation. Gather-and-sort and render are unchanged, as they must be, since only the source of the particle state differs. Running the comparison the other way—holding *wall-clock* fixed instead of count, 215 vortons per slab sized from the measured $2.8\times$ ratio—the 3D solver still reaches spectral distance 0.23 with $\rho=0.77$: a better in-plane spectrum than the shipping stack at the same coherence. What it cannot do at that budget is *be the picture*: 1.9k renderable Gaussians no longer tile the flame that the stack’s

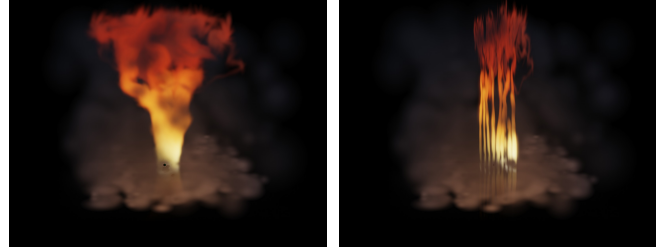


Fig. 8. Slice orientation. Same simulation state, seed, camera and shading; only the slice frame differs. The camera sits at a 79° orbit—close enough to the slice planes’ edge to expose the degeneracy, oblique enough that the individual planes are still distinguishable. *Left*: view-aligned slices re-orient with the camera, so the stack integrates into a volume and reads correctly from any azimuth. *Right*: world-fixed slices seen near edge-on separate into the flat sheets they actually are—the plume loses its silhouette and becomes nine ribbons standing on the ground. At exactly 90° they collapse further, into single-pixel lines. This is the failure mode that motivates view alignment, and the reason a Lagrangian state that can be re-sliced per frame at no re-simulation cost is worth having.



Fig. 9. Inter-layer coupling (Eq. 7). *Left*: $c=0$, independent slices—the stack reads as several superimposed fires with uncorrelated sway. *Right*: $c=1$, slices slaved to the coarse solver—one coherent but overly smooth column. The default $c=0.5$ (Figure 1) preserves per-slice turbulence on a shared bulk motion. Snapshot field-correlation probes proved statistically unstable for quantifying either coupling mechanism (§7); the comparison here is deliberately qualitative.



Fig. 10. Covariance dynamics. *Left*: isotropic splats ($k_\Sigma=0$) read as soft blobs. *Middle*: flow-driven covariance (Eq. 9) shears splats into flame licks. *Right*: adding the trajectory filament stretch (Eq. 10) elongates fast particles into path-aligned streaks.

5.4k do, and the deficit is exactly the image-quality criterion that set N_p in §7. The stack’s particle budget is a rendering constraint as much as a solver one; a system free to simulate on one set of elements and render on another would make the equal-cost 3D solve the better choice, and §8 lists that decoupling as future work.

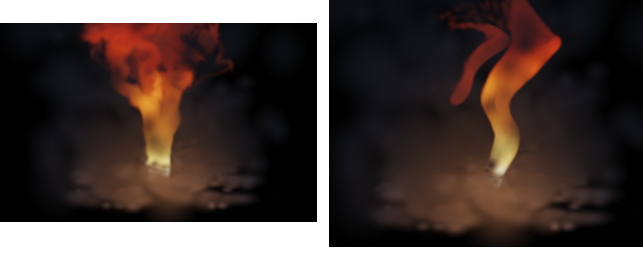


Fig. 11. Layered 2.5D vs. a matched-budget kinematic 3D mode. *Left*: nine coupled 2D vortex-particle slices (default). *Right*: the same total particle budget advected by analytic 3D curl noise with entrainment—a volumetric alternative that lacks the baroclinic rolling produced by the slice solvers. The *quantitative* comparison against a genuine 3D reference solve is Figure 14; this panel is the qualitative matched-budget rendering ablation.

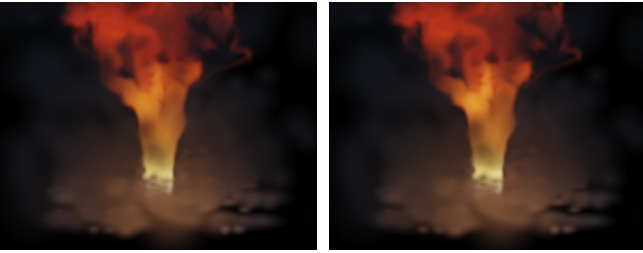


Fig. 12. HDR post ablation: without (left) and with (right) bloom before ACES tone mapping. The prefilter threshold admits only the emission-dominated core, so bloom is deliberately a small, local effect—it changes 1.5% of pixels—adding the incandescent halo around the white-hot base without lifting the plume as a whole. The threshold is a real dial, not decoration: set above the scene’s peak HDR value it admits nothing and bloom becomes a no-op.

We draw the honest conclusion. *The layered decomposition is not what makes this system real-time*—a matched 3D solve is under a factor of three away and remains interactive at the shipping budget. What layering buys is resolution allocation: in-plane detail is cheap and depth is charged only for the slices actually rendered, which is why the architecture scales to the slice counts production layered fire uses [?]. What it costs is measured next, and having this solver in-process lets us ablate the term the stack cannot carry rather than argue about it (Figure 13).

Spectral comparison against a true-3D reference. Figure 14 confronts the layered approximation with a matched true-3D reference: a $32 \times 48 \times 32$ Boussinesq Euler solve (semi-Lagrangian advection, Jacobi pressure projection, seeded symmetry-breaking forcing at the source; solver in supplemental) with the same domain, cell size, heat source, cooling, and Δt as the slice stack, compared on the same observable—the in-plane (u, v) fluctuation field on $x-y$ planes sampled at the nine slices’ exact world depths. The stack is exported from the shipping solver with procedural turbulence and wind disabled, isolating solver dynamics; both sides use the replay protocol (40 snapshots after warm-up, seeded). Three findings. (i) *The grid-free stack recovers most of the reference’s spectral shape—a gap that was previously dominated by the grid.* The 3D reference distributes

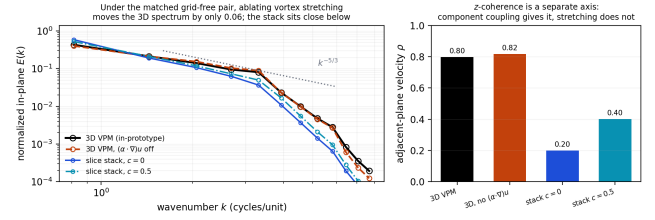


Fig. 13. Ablating the layered decomposition’s missing term inside an otherwise identical solver (§7). The 3D Gaussian vortex-particle solver runs in the shipping prototype and shares the kernel family, core radius, opening angle, particle scheme, source terms, timestep, mapping and renderer with the slice stack, and is sampled on the same nine planes through the same protocol. *Left*: under the grid-free pair the stack’s spectral gap to matched 3D is modest ($\log\text{-}L_1$ 0.28–0.40), and switching off vortex stretching $(\alpha \cdot \nabla)u$ inside the 3D solver moves its spectrum by only 0.06—under this discretization, most of the residual gap is the loss of three-component transport coupling, *not* the stretching cascade. (The retained grid-based pair had attributed $\sim 70\%$ of a far larger gap to stretching: the attribution is discretization-dependent, which is itself a finding.) *Right*: z-coherence remains the stack’s clearest deficit: adjacent-plane $\rho = 0.80$ for 3D against 0.20 (uncoupled) / 0.40 (coarse-coupled) for the stack; the inter-slice exchange of Eq. 8 recovers most of it (§7). Any future inter-slice operator must be measured on both axes.

energy broadly (48% in the largest-scale bin; $\sim 30\%$ in the mid-band $k \in [2, 4]$ cycles/unit, tracking $k^{-5/3}$). The grid-free stack configurations sit at 50–67% largest-bin (per-config means over four seeds; seed spread within ± 3 points) and 15–26% mid-band—log-spectral distance 0.39–0.68, seed ranges within ± 0.03 of each mean—where the retained vortex-in-cell stack, measured through the identical protocol, concentrated 83–92% in the largest bin with only 2–5% mid-band (distance 0.78–0.84). The interpretation is direct: with the baroclinic source evaluated on the particles rather than through a 32×48 grid, the mid-band rolling that the grid’s Nyquist suppressed is actually simulated. This is the measured payoff of grid freedom, purchased by the $2.3\times$ simulation step of Table 4. (One sampling caveat, stated once: the grid-free observable is a point-sampled continuous field, so a small tail of above-Nyquist kernel energy aliases into the export; at $\sigma_0 = 0.05$ the kernel retains $\sim 4\%$ amplitude at the sampling Nyquist, far too little to explain a mid-band that moved from 3% to 26%.) Density remains the knob to check: the overlap condition $h \lesssim \sigma$ is satisfied at 600 particles per layer ($h = 0.018$ in the plume interior, $h/\sigma_0 = 0.37$), and repeating the entire measurement at 2600 per layer ($h/\sigma_0 = 0.25$) improves the distance moderately ($0.46 \rightarrow 0.38$ mean log-spectral distance, a 17% reduction; best condition 0.31, largest bin 0.50 against the reference’s 0.48, mid-band 0.28 against 0.30)—the result is near-converged in N_p , and refinement moves it *toward* the reference, not away.

Three further controls close the remaining confounds, and one of them cuts against us before it cuts for us. (a) *Grid refinement.* The obvious rejoinder—“give the grid more cells”—we test directly: at 64×96 per slice (iteration count scaled with resolution) the vortex-in-cell stack recovers most of the in-plane spectral shape, as doubling the Nyquist must (log-distance 0.19 uncoupled, 0.41 coupled; three

seeds). What it does not recover is coherence: its grid-form coupling reaches only $\rho=0.41$ where the pairwise particle exchange reaches 0.76, and its cost, 33.4 ms per frame at 9×600 , already exceeds the grid-free solver's 29.8 ms at 9×1300 . (b) *The core radius, swept.* The grid-free solver's own resolution dial costs nothing to turn: at $\sigma_0=0.035$ (overlap ratio $h/\sigma_0 \approx 0.51$, still tiled; simulation cost unchanged) the spectral distance drops to 0.16 with $\rho=0.61$ retained. On the three axes together—spectral shape, z-coherence, milliseconds—no grid configuration we measured dominates this setting; the finer grid buys shape alone, at higher cost, with less than half the coherence. The shipping $\sigma_0=0.05$ is chosen for image quality (blob coverage), not spectral fidelity, and the dial is free to re-turn per shot; at $\sigma_0 \geq 0.07$ the field over-smooths rapidly (distance 1.5–2.3, adjacent slices nearly identical at $\rho \rightarrow 0.99$), so the usable band is real but narrow. (c) *Timestep.* Repeating the full measurement at $\Delta t = 1/30$ and $1/120$ s moves the distance by less than 0.03 in either direction (0.59/0.58/0.55)—the observable is timestep-converged.

The remaining confound is implementation: the reference is separate NumPy code. The in-prototype 3D vortex-particle control (§7) removes it—same kernel, same σ_0 , same θ , same particle scheme, sampled through the same export protocol—and independently reproduces the external reference's character (largest bin 0.43 vs. 0.48; mid-band 0.34 vs. 0.30), corroborating both. It also prices the one term the stack structurally cannot have: *switching vortex stretching off inside the matched 3D solver moves its spectrum by only 0.06* (largest bin $0.43 \rightarrow 0.40$, mid-band $0.34 \rightarrow 0.37$), against a stack-to-3D distance of 0.28–0.40. Under this discretization the missing stretching cascade explains only $\sim 20\%$ of a much smaller gap; the rest is the loss of three-component transport. We note plainly that the corresponding grid-based control had attributed $\sim 70\%$ of its (four times larger) gap to stretching: the attribution is discretization-dependent, and the grid-free pair is the better-matched control for the shipping solver. (ii) *The two couplings now trade off measurably* (four-seed means): coarse coupling improves spectral shape (distance 0.53 uncoupled $\rightarrow$ 0.39 at $c=0.5$), while the inter-slice exchange slightly smooths it ($0.53 \rightarrow 0.68$ at $\kappa_z=0.45$; $0.39 \rightarrow 0.54$ with both) —a diffusion, as its construction says it is—and buys coherence instead. (iii) *z-coherence, the stack's clearest deficit, is now largely recoverable.* Adjacent reference planes correlate at $\rho \approx 0.93$ and the matched 3D control at 0.80; the uncoupled stack sits at 0.18 (four-seed mean; range 0.11–0.22). Coarse coupling raises it to 0.29 (0.19–0.38, the most seed-sensitive number we report); the particle-strength-exchange operator of Eq. 8 raises it to 0.62 alone and 0.76 combined (range 0.66–0.82; $\rho(\Delta z)$ RMS distance to the reference 0.15)—where the same operator in grid form had measured an inconclusive 0.13–0.19 at 32×48 and reaches only 0.41 even at 64×96 . The pairwise exchange acts on the circulations the induced field is actually made of, rather than on a resampled vorticity grid, and the coherence it produces survives the Biot–Savart reconstruction. The spectral price in (ii) is the flip side; the two axes must be read together, and neither implies the other. The coherence is also directly visible: Figure 15 shows the same seed at a 63° world-fixed side view with the operator off and on—independent per-slice flames, each with its own hot core, fuse into a single connected volume. These curves make §8's honesty quantitative—and they are the explicit target any future inter-slice coupling must be measured against.

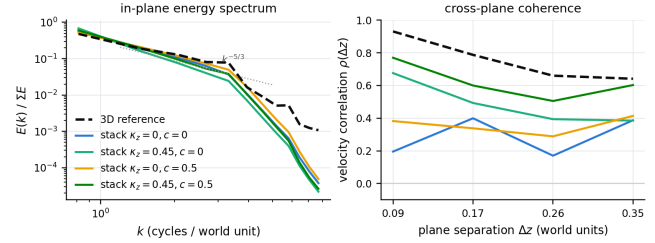


Fig. 14. The layered approximation, measured against a matched true-3D Boussinesq reference (dashed). *Left:* normalized in-plane energy spectra. The 3D reference spreads energy across scales ($\sim k^{-5/3}$ through the mid-band); the grid-free stack now tracks it through the mid-band (largest bin 0.50–0.67 across configurations, four-seed means, vs. 0.48; coarse coupling c sharpens the spectrum, the inter-slice exchange κ_z smooths it), a regime the retained grid-based stack never reached (0.83–0.92 largest-bin). Curves are exported at 600 particles per layer; repeating the measurement at 2600 moves them moderately toward the reference (§7). *Right:* cross-plane velocity correlation at the slices' depths. The 3D flow is strongly z-contiguous ($\rho \approx 0.93$ adjacent); the uncoupled stack is not (0.18), and the particle-strength-exchange operator recovers most of the gap (0.62 alone, 0.76 with c ; four-seed means—the curves shown are the seed-7 member). Correlations shown for $\Delta z \leq 0.35$; larger separations in the reference are contaminated by the source's mirror symmetry (supplemental).

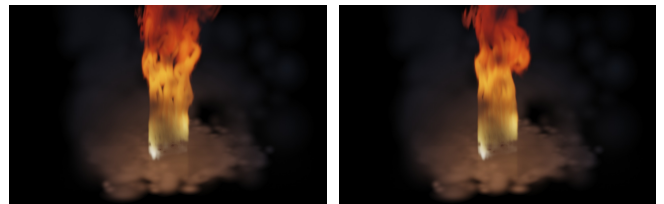


Fig. 15. The inter-slice exchange operator, seen rather than plotted: same seed, same step count, world-fixed slices from a 63° side view (turbulence, wind and coarse coupling disabled to isolate the operator). *Left* ($\kappa_z=0$): each slice burns as an independent flame with its own hot core—the stack reads as separate sheets. *Right* ($\kappa_z=0.45$): the pairwise circulation exchange fuses the bases into one core and the bodies into one connected volume, the visual counterpart of ρ : 0.18 $\rightarrow$ 0.62 (§7).

The reference itself is a modest solve ($32\times 48\times 32$, Jacobi projection); we use it as a matched control, not as a converged ground truth, and say so.

Scene occlusion. Figure 7 exercises the compositing claim directly, against a ladder of three baselines so that no panel is broken for an unrelated reason. The *depth-aware* baseline (top right) is the strongest alternative we know how to build without interleaving: scene splats of footprint opacity ≥ 0.5 write a depth buffer and fire fragments are z-tested against it, the natural upgrade of backdrop compositing [?] to per-pixel occlusion. It handles opaque occluders, and its remaining failure is structural rather than a tuning artifact: at every semi-transparent boundary the z-test is binary, so the soft Gaussian interface between flame and fire ring collapses into an aliased silhouette, and splats near the surface pop across it. The *backdrop* baseline (bottom left) removes the depth test and shows

the flame floating in front of every primitive; the *unsorted* panel (bottom right) additionally drops the intra-scene sort, which we report separately rather than let that defect stand in for the prior art. The interleave’s overhead is small: 30.7 vs. 28.3 ms median render time at 45.8k scene splats over the full 9×1300 flame (+8%, within the run-to-run spread; same session as Table 4).

Ablations. Figures 6–12 ablate the major components: simulated vs. procedural sources, coupling strength, covariance dynamics, the layered representation itself, and HDR post. Each toggle is live in the interactive prototype.

Interactive control. Because dynamics are forward-simulated, the system responds immediately to user input: shift-dragging moves the fire source and the plume follows with vortical lag; wind tilts the column; the coupling, buoyancy, baroclinic, and cooling gains are all live sliders; and fire and smoke are two shading presets over the same particle state, switchable per keystroke.

8 Limitations and Future Work

Physics of stacked 2D slices. Vortex stretching $(\omega \cdot \nabla)\mathbf{u}$ vanishes in 2D, and 2D turbulence cascades energy toward *large* scales, opposite to 3D; a slice stack therefore cannot reproduce the statistics of three-dimensional fire turbulence, however plausible it looks. We have *measured* what remains of this gap under the grid-free solver (Figure 14): the stack holds 50–67% of in-plane energy in the largest-scale bin where a matched 3D reference holds 48% (log-spectral distance 0.28–0.40 to the matched in-prototype control), and uncoupled adjacent-plane coherence is 0.18 vs. the reference’s 0.93—recoverable to 0.76 by the inter-slice exchange (Eq. 8), which reintroduces the dropped out-of-plane *transport* at a measured spectral-smoothing price. What no exchange supplies is vortex-stretching *amplification*; under the matched grid-free control its ablation prices at ~20% of the residual spectral gap, the rest being three-component transport itself. We deliberately trade the remaining fidelity for interactivity in a regime—turbulent, shapeless, emission-dominated media—where the eye is forgiving: visual plausibility rests on the large and mid scales the grid-free stack does capture plus appearance-side detail (FBM footprints, flow-driven covariance, bloom), not on reproducing full 3D statistics. Figure 14 is the standing target for any future coupling design.

Illumination. Fire-to-scene lighting is a screen-space approximation, not transport: Gaussian scene splats are warmed by proximity to the flame’s image-space position. Correct volumetric emission from Gaussian mixtures is available in a ray-traced formulation [?]; a hybrid that shades nearby scene splats from the flame’s particle set (a few thousand point lights, amenable to clustering) is future work, as are cast shadows and smoke self-shadowing.

Renderer completeness. Our rasterizer implements the forward 3DGS math (sort, EWA projection, footprint, emission-absorption blend; degree-1 spherical harmonics for loaded scenes) but not the differentiable backward pass—our demo has no reconstruction objective—and fire splats themselves carry no view-dependent color, which isotropic incandescence does not need. The FBM modulation of the splat footprint also departs from the pure Gaussian kernel, so

fire splats exported to a stock 3DGS viewer would render softer than shown here; an export-compatibility experiment would sharpen the GS-native claim. The exact CPU sort of the dynamic set is adequate at 10^4 Gaussians and the $O(n)$ scene bucketing scales to 10^5 (§5.5), but ordering between the streams is bucket-granular, and tile-binned GPU sorting remains the right implementation for dense fire inside full reconstructed scenes.

View dependence and multi-view consistency. View-aligned slices make the fire’s world-space geometry a function of the rendering camera: the same particle lifts to different world positions as the camera orbits, so two simultaneous viewers—or the two eyes of a stereo pair—observe slightly different fires. This is the sharpest tension in the “GS-native” framing: the fire’s *attributes* are camera-independent, but its slice lift is view-adapted, unlike a static reconstruction. (Solver-side grid freedom does not dissolve this tension—representation and view dependence are orthogonal axes, and we claim progress only on the former.) The world-fixed mode removes the dependence at the cost of edge-on collapse (Figure 8); for stereo, sharing one lift per eye pair is a cheap fix. More fundamentally, because particles are world-space objects, slices can be re-binned to a new axis without restarting the simulation—a remedy unavailable to raster-state slice methods, which we have not yet implemented. Very fast camera motion likewise changes the decomposition axis mid-shot, and flows predominantly along the view axis remain the weakest case [?].

Radiometric level of detail. In the optically-thin, emission-dominated regime the front-to-back accumulation of Eq. 11 is approximately additive, so distant fire clusters could be merged into fewer Gaussians while conserving the summed emission $c\alpha$ and its emission-weighted second moment—a radiance-conserving LOD criterion, distinct from the photometric-loss LOD of opaque-splat hierarchies [?]. This is the concrete route to dense fire inside large reconstructions; we have not implemented it.

Geometry-aware flow. The loaded scene could also act on the *simulation*, not only the compositing: sampling the reconstruction’s surfaces into boundary vortex Gaussians whose strengths cancel the normal flow—a panel method, or the method of images our ground mirror already instantiates (§4.2)—would deflect the flame around the captured geometry in real time, staying grid-free: the per-slice, Lagrangian counterpart of the occupancy-grid obstacle boundaries that offline scene-combustion imposes [?].

Toward reconstruction. The mapping of Table 2 is differentiable end-to-end in principle. Because the forward model exposes only a handful of interpretable controls (source, wind, buoyancy, baroclinic and coupling gains, cooling), fitting *those* to a video summary statistic—a far lower-dimensional, better-constrained inverse than fitting a free field or free Gaussians [??], and solvable even with gradient-free system identification [?—would recover not a frozen radiance field but a *running, editable* simulation. Demonstrating a single fitted example is the natural next step.

What this evaluation does not yet cover. We state the gaps explicitly rather than leave them to be discovered. (i) *Real captures.* Every scene-side result—occlusion, in-scene ignition, spread—uses

a synthetic .ply (45 808 splats), a more favorable input for both depth bucketing and a splat adjacency graph than a real capture would be. The .ply path itself implements the standard 3DGS forward projection—the full 3×3 covariance is transformed through the view rotation and perspective Jacobian (EWA [?]), the screen-space conic’s eigenvectors span the quad, and degree-1 spherical harmonics supply view-dependent color—so what remains outstanding is the input, not the projection: results on public captured reconstructions are the gap we most want closed. (i b) *Solver refinement*. The particle-density, core-radius, grid-resolution and timestep confounds on the spectral result are all now swept directly (§7), as is the treecode opening angle θ against the exact sum; the remaining unswept axis is hardware—all timings come from one workstation, and a second, weaker machine (integrated GPU) is the measurement we would add first. (ii) *Baselines*. The comparison we consider most informative—a true 3D solver rendered through *our own* pipeline, isolating the layered decomposition from every other difference—is now implemented and reported (§7). We still report no head-to-head against another *system*: the published numbers we cite were measured on different hardware, at different resolutions, and with far larger simulation state, so we use them to place the design point rather than to claim a speedup. (iii) *Hardware*. All timings come from a single workstation; a low-end second point would bound the interactive claim. (iv) *Ablations*. A slice-count sweep at fixed total particle budget, and a non-saturating viewpoint-isotropy measure swept densely in azimuth, would replace the qualitative arguments in §4.3. (v) *Perceptual evidence*. Our central trade—physical fidelity for interactivity in a regime where the eye is forgiving—is an empirical claim about viewers that we argue for but do not test.

9 Conclusion

We have presented a system in which fire and smoke live natively inside Gaussian-splatting scenes: not reconstructed, not ray-marched over a backdrop, but forward-simulated in real time by vortex particles that *are* the rendered Gaussians—now in the strict sense: the solver evaluates its own velocity, gradient, and temperature fields in closed form from the same means, widths, and amplitudes the rasterizer draws, with no grid anywhere. Its most distinctive behavior is a closed loop—the flame ignites the reconstruction, combustion state lives on the scene’s own splats, and burning geometry re-seeds the solver, so that what burns next is decided by the flow and not only by the adjacency structure. The enabling observation, that Lagrangian vortex elements and 3D Gaussians are the same class of point primitive, collapses the simulation-to-rendering pipeline to a set of closed-form functionals of one Gaussian population; a solver whose entire state is the splats themselves—no pressure solve, no linear solve, no field to resample—is what keeps that loop inside an interactive frame budget. We have also measured what the layered decomposition costs, against a true 3D solver inside the same prototype and behind the same renderer, and used it to ablate the term the layering cannot carry rather than argue about it. Having demonstrated the pattern across flame, smoke, and embers, we conjecture that the same particle-set unification extends to other optically thin, surfaceless Lagrangian phenomena (sparks, spray,

dust, aurorae), and that GS-native effects are a necessary ingredient if reconstructed scenes are to become fully dynamic worlds.

A Per-Slice Solver Pseudocode

```

step_all(dt):
    # once per frame
    for each sim layer L:
        # Phase A: emit + pack + build
        emit at source (budgeted)
        for particle i in L:
            # dynamics temperature
            if in source band: T_i += dt*s0*exp(-ks*x^2)
            T_i *= (1 - 0.5*kc*dt); clamp
            if nu > 0: sig_i^2 += 2*nu*dt # core spreading
        Morton-sort particles; pack SoA in sorted order
        build quadtree; per node: signed Gamma, |Gamma|-centroid,
        signed dipole, matched sig_c^2, MAC radius
        director = merge top tree levels across layers
        + ground-mirror images (x,-y,-Gamma)
    if kz > 0: pairwise exchange between adjacent layers:
        d = dt*kz*eta_ij*(G_j - G_i); G_i += d; G_j -= d

step_slice(L, dt):
    # Phase C: N-body + update
    for particle i:
        traverse tree once, accumulating in closed form:
        u_i, grad-u L_i # Lamb-Oseen kernel + analytic gradient
        S=sum w, sum w*T, moment-renormalized dT/dx # mixture T
        T~ = (sum w*T)/S
        dTdx = sum((T_j-T_i)(x_j-x_i)w) / sum((x_j-x_i)^2 w)
        v = u_i + beta_b*T~ * y-hat # direct buoyancy
        v += c*(U_director(x_i) - v) # nudge (Eq. couple)
        v += turb*curl_noise(x_i,t) + wind; clamp |v|
    for particle i:
        # integrate (frame-lagged baroclinic)
        G_i = (1-lam*dt)*G_i + dt*beta*dTdx_i; clamp
        Sigma_i evolved by L_i # rendered covariance (Sec. 5.2)
        x_i += dt*v_i
        Tc_i -= dt*kappa*Tc_i^2 # color temperature (render)
        d_i *= (1 - 0.3*dt)
        recycle if cold/exhausted/out of bounds

```

B Supplemental Material

Video (65 s). Four single-take sequences, unedited. The first three are *side-by-side solver comparisons*: the retained vortex-in-cell baseline on the left and the grid-free solver on the right, run from the *same seed* at the *same fixed timestep* through the identical renderer and shading, so every visual difference is attributable to the solver—(1) fixed camera, (2) a constant-rate orbit, and (3) interaction (scripted source drag, wind, and the fire/smoke re-shading of one particle state, the same scripted inputs applied to both sides). Sequence (4) shows the fire-spread loop on a loaded scene, in which burning splats feed the solver and the plume above the fire is sustained by the scene. All sequences run the shipping configuration ($N=9 \times 600$). Frames are rendered deterministically at $\Delta t = 1/60$ s from per-sequence seeds and encoded at 60 fps, so playback speed equals simulation speed; each side’s frame-time overlay shows the rate measured for *that* solver on the reference machine (15.5 ms/65 FPS grid, 17.1 ms/58 FPS grid-free), consistent with Table 4. The interactive analogue of the comparison sequences ships with the demo: a lockstep side-by-side page driving two instances from one clock, seed and control stream.

Code. The complete system is the single dependency-free HTML file released with this submission—including the retained vortex-in-cell baseline (with the runtime-selectable 64×96 control resolution), the exact-Biot–Savart reference path, and the in-browser benchmark/treecode-error harness—together with the 3D reference

solver, the spectral-analysis scripts (single- and multi-seed, including the grid-refinement, core-radius and timestep controls of §7), and the scene generator used for the composition results.